



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Control Engineering and Information Technology

Algorithms for Real-Time Physics Simulation

BACHELOR'S THESIS

Author
Márton Biró

Advisor
dr. Márton Vaitkus

May 29, 2026

Contents

Kivonat	i
Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Contribution	2
1.3 The primal/dual lens	2
1.4 Evaluation axes	3
1.5 Thesis structure	3
2 Background and theoretical foundations	4
2.1 Time-stepping schemes for physical simulation	4
2.1.1 Explicit time integration (forward Euler method) and why it fails . .	5
2.1.2 Implicit time integration (backward Euler method)	5
2.2 Implicit Euler as energy minimization	5
2.3 Mass-spring systems	6
2.3.1 Limitations of mass-spring models	6
2.4 Solver structure: Gauss–Seidel vs. Jacobi updates	6
2.5 The primal/dual distinction	6
3 Related work	8
3.1 From PBD to XPBD	8
3.2 Vertex Block Descent (VBD)	9
3.3 Prior comparison: Ma et al. (2024)	9
3.4 Reference materials	10
4 The two algorithms	11
4.1 XPBD: dual-variable position projection	11
4.1.1 Derivation	11

4.1.2	Algorithm	12
4.1.3	Characteristics	12
4.2	VBD: primal block-coordinate descent	13
4.2.1	Derivation	13
4.2.2	Algorithm	13
4.2.3	Adaptive initialization	13
4.2.4	Characteristics	14
4.3	Side-by-side: where the algorithms differ	14
5	Methodology and implementation	16
5.1	The Unity game engine	16
5.1.1	Pinning vertices through Unity components	16
5.2	Platform and environment	17
5.3	Integration architecture	17
5.3.1	Cloth model construction	17
5.4	How the comparison is kept fair	18
5.4.1	Stiffness $\leftrightarrow$ compliance mapping	18
5.4.2	Matched compute budget — and the substep/iteration caveat	18
5.4.3	Matched gravity, dt , and damping	18
5.4.4	Handling VBD’s accelerations for the core comparison	18
5.4.5	Hessian projection for stability	19
5.5	Self-collision handling	19
5.6	Parallelization (forward-looking)	19
6	Experiments and results	20
6.1	Computational efficiency	20
6.1.1	Influence of substep and iteration count on error and compute time .	20
6.1.2	Frame time vs. node count	21
6.2	Robustness and stability	22
6.2.1	Chain with heavy weight attached	22
6.2.2	Chain with high stiffness ratio	23
6.2.3	Self-collision: cloth folded onto itself	23
6.3	Physical realism	24
6.3.1	Harmonic oscillator	24
6.3.2	Energy tracking	24
6.4	Flexibility	25
6.4.1	Iteration-count independence of stiffness	25

6.4.2	Ease of adding a new functionality	26
6.4.3	Coverage of different physical domains	27
7	Discussion	28
7.1	Synthesis across axes	28
7.2	Verdicts across the four axes	28
7.3	Limitations and threats to validity	30
7.3.1	Single-threaded execution	30
7.3.2	Mass-spring discretization only	30
7.3.3	Gauss–Seidel order dependence	30
7.3.4	Canonical 2024 VBD only — the AVBD caveat	30
8	Conclusion and future work	31
8.1	Summary	31
8.2	Future work	31
8.2.1	Multithreaded and GPU comparison	31
8.2.2	Extending the research to other domains of dynamics	32
8.2.3	Using a different cloth model	32
8.2.4	AVBD and post-2024 variants	32
	Acknowledgements	33
	Bibliography	34
	Appendix	36
A.1	Declaration on the Use of Generative Artificial Intelligence	37
A.2	Full solver pseudocode	38
A.2.1	XPBD, full implementation	38
A.2.2	VBD, full implementation	39

STUDENT DECLARATION

I, *Márton Biró*, the undersigned, hereby declare that the present Bachelor's Thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized. In the Appendix, I have clearly indicated whether I used artificial intelligence tools in preparing the thesis; if so, I have specified the manner and extent of their use in the table. I acknowledge that I bear full responsibility for any content generated by artificial intelligence – regardless of the extent of its use.

I agree that the basic information of my present work (author(s), title, abstracts in English and in a second language, year of preparation, name of advisor(s)) will be made publicly available on the internet by BME VIK, and the full text of the work will be available to persons registered in the BME Directory. I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, May 29, 2026



Márton Biró
student

Kivonat

A valós idejű deformálható testek szimulációjának néhány milliszekundumon belül kell hihető dinamikát szolgáltatnia képkockánként, ami minden *solver*t a fizikai pontosság és a számítási költség közötti kompromisszumra kényszerít. Az Extended Position-Based Dynamics (XPBD) évek óta a valós idejű deformálható testek solveinek egyik legelterjedtebb változata, míg a 2024-ben bemutatott Vertex Block Descent (VBD) egy ígéretes alternatíva, amelyet még nem hasonlítottak össze alaposan az XPBD-vel. Ez a szakdolgozat a két módszer első, az eredeti cikkekhez hű, közvetlen összehasonlítását mutatja be.

Mindkét *solver*t ugyanabban a környezetben valósítom meg, ugyanazokra a modellekre alkalmazom, és egyező számítási költséggel futtatom, hogy a wall-clock költségük közvetlenül összehasonlítható legyen. Egy magas iterációs számú Newton *solver* szolgál ground-truth referenciaként, így nemcsak a relatív teljesítményről, de az abszolút megoldási hibáról is be tudok számolni. Az eredményeket Macklin et al. primal/dual keretrendszerén keresztül vizsgálom, amely a két módszert ugyanazon probléma különböző irányból történő megközelítésének tekinti, és segít az eredmények értelmezésében.

Mindkét *solver*t négy szempont alapján értékelem ki: hatékonyság, robusztusság, fizikai valósághűség és rugalmasság. A fő megállapítás az, hogy egyik módszer sem dominál; mindegyik ott teljesít jobban, ahol a frissítési iránya illik a feladathoz. A pontosság–költség kompromisszum terén a relatív helyzetük a részlépésenkénti feladat nehézségével változik, és mindkét módszer jellegzetes erősségeket és hibamódokat mutat terhelés alatt. Elegendően magas iterációs számnál mindkettő eléri a Newton-referenciát, és fizikailag megkülönböztethetetlené válik egymástól.

A gyakorlati ajánlás tehát feladatfüggő, nem abszolút, és az eredmények arra utalnak, hogy a többszálú vagy GPU-s végrehajtás — amelyet az AVBD-vel és alternatív szövetmodellekkel együtt jövőbeli munkának hagyok — tovább billentené a mérleget a VBD irányába.

Abstract

Real-time deformable simulation has to deliver believable dynamics within a few milliseconds per frame, which forces every solver onto a trade-off between physical accuracy and computational cost. Extended Position-Based Dynamics (XPBD) has been the workhorse of real-time solvers for years, while Vertex Block Descent (VBD), introduced in 2024, is a promising alternative that has not yet been thoroughly evaluated against it. This thesis presents the first faithful head-to-head comparison of the two.

I implement both solvers in a shared codebase, apply them to the same models, and run them under matched computational budgets so that their wall-clock cost is directly comparable. A high-iteration Newton solver provides the ground-truth reference, letting me report absolute solution error rather than only relative performance. Throughout, I read the comparison through the primal/dual lens of Macklin et al., which frames the two methods as approaching the same problem from different directions and helps interpret the results.

I evaluate both solvers along four axes: efficiency, robustness, physical realism, and flexibility. The headline finding is that neither dominates; each wins in the setting its update direction suits. On the accuracy–cost trade-off their relative standing shifts with the difficulty of the per-step problem, and each method shows characteristic strengths and failure modes under stress. With enough iterations both converge to the Newton reference and become physically indistinguishable.

The practical recommendation is therefore scenario-based rather than absolute, and the results suggest that multithreaded or GPU execution — left to future work, together with AVBD and alternative cloth models — would shift the balance further toward VBD.

Chapter 1

Introduction

1.1 Motivation

Real-time simulation has become a baseline expectation in games, virtual reality, and augmented reality. A simulator must deliver believable dynamics within a tight frame budget: at 60 fps, only about 16 ms is available for all physics, rendering, and gameplay logic combined, and physics receives only a fraction of that.

This thesis focuses on deformable bodies — springs, chains, and cloth — rather than rigid bodies, for several reasons. Deformable simulation is the harder case, with many more degrees of freedom and constraints per frame, so it stresses both solvers more heavily. Even when vertices and constraints number in the tens of thousands, every part of the simulated system remains visible, so solver behavior can be observed directly and failures are easy to spot. Springs, moreover, are among the most common and most easily measurable primitives in simulation, while cloth is widely used in practice and a crucial stepping stone toward full soft-body simulation.

Designing a real-time simulator of this kind is hard for three recurring reasons. The first is *stiffness*: cloth and soft tissue impose very stiff constraints — almost-inextensible fabric, near-rigid bone — and stiff systems make explicit integrators unstable unless the time step shrinks to impractically small values. The second is *stability*: even implicit integrators can blow up under poorly conditioned constraints, large mass ratios between connected particles, or the velocity discontinuities introduced by contact. The third is the *strict time constraint*: methods that converge well offline, such as Newton’s method or projective dynamics, are too expensive per frame, while cheap real-time methods typically trade convergence quality for predictable cost.

The accuracy/efficiency trade-off. Taken together, these pressures place every physics-based simulator somewhere on a line between *physical accuracy* (combining stiffness and stability) and *computational efficiency*; pushing toward efficiency inevitably costs realism. Table 1.1 sketches this spectrum for the best-known deformable-body solvers, ordered from most accurate to most efficient.

At the efficient extreme, the Mass-Spring System (MSS) represents a deformable material as a network of point masses connected by springs and advances it with explicit Euler integration — the most intuitive and straightforward approach. Although it is in principle more physically grounded than PBD or XPBD, it is highly unstable for stiff and complex systems, which pushes it to the bottom of the list: explicit integration of stiff cloth requires

impractically small time steps to stay stable, which is exactly what motivated the move to implicit integration [1]. At the accurate extreme, the Finite Element Method (FEM) treats an object as a continuous material discretized into volumetric finite elements (such as tetrahedra) and solves the resulting implicit-Euler problem with a Newton solver. It excels at physical accuracy, but its high computational cost makes it impractical for real-time applications. [12]

Method	Accuracy	Cost
FEM	reference	most expensive
VBD	moderate-high	moderate
XPBD	moderate	cheap
PBD	low (stiffness-dep.)	cheap
MSS	poor (unstable)	cheapest

Table 1.1: Deformable-simulation methods ordered from most accurate to most efficient. XPBD and VBD are in the real-time domain.

XPBD and VBD both sit near the efficient end of this spectrum, inside the real-time category. XPBD [8] has been a workhorse for years, whereas VBD [3] is a recent and promising addition that has not yet been thoroughly evaluated head-to-head against XPBD.

1.2 Contribution

The central contribution of this thesis is, to the best of my knowledge, the *first faithful XPBD-vs-VBD comparison*. Both algorithms are implemented in the same Unity-based codebase, applied to the same mass-spring material model, and run under a matched compute budget — matched iteration and substep counts, not merely matched material parameters. They are evaluated not only qualitatively but also against a high-iteration Newton solver that serves as a numerical ground-truth reference, which lets me report *solution error* rather than only relative performance.

Crucially, the comparison uses XPBD — the constraint-stiffness-independent variant that any practitioner would deploy today — together with the canonical VBD formulation, rather than plain PBD or a non-canonical variant. This sharpens the closest prior comparison, Ma et al. [6], which contrasts plain PBD with a non-canonical VBD implementation and reports only frame-rate scaling; Section 3.3 sets out exactly how my setup differs.

1.3 The primal/dual lens

The two solvers sit on opposite sides of the *primal/dual* duality identified by Macklin et al. [7]: XPBD is the dual solver, solving for Lagrange multipliers on the constraints, while VBD is the primal solver, solving directly for vertex positions. This duality is the organizing lens of the thesis: it predicts complementary weak spots — one per solver — that the stress tests in Chapter 6 are built around. Section 2.5 derives the distinction and the failure-mode predictions in full.

1.4 Evaluation axes

I evaluate both solvers along four axes that together capture the trade-offs a practitioner cares about:

Efficiency. Computational cost per frame as a function of mesh size and substep/iteration count.

Robustness. Behavior under stress: extreme mass ratios, extreme stiffness ratios, and the stability issues they produce.

Physical realism. Energy behavior, damping behavior, and how close each method comes to a ground-truth Newton reference.

Flexibility. The independence of material properties from iteration count, the ease of adding new constraint types, and the domain coverage of each model.

1.5 Thesis structure

The remainder of the thesis is organized as follows. Chapter 2 lays the theoretical foundation: the explicit and implicit time-stepping schemes, the variational form of implicit Euler that unifies both solvers as approximate minimizers of the same energy, the mass-spring model, the Gauss–Seidel structure shared by both solvers, and the primal/dual distinction that organizes the comparison. Chapter 3 traces the PBD-to-XPBD and VBD lineages and positions this work against the closest prior comparison, Ma et al. [6]. Chapter 4 states both algorithms in their canonical core-comparison form — derivation, pseudocode, and characteristics — and contrasts them side by side. Chapter 5 describes the Unity implementation, the cloth model, and the fairness controls that keep the comparison balanced, along with self-collision handling and the parallelization outlook. Chapter 6 presents the results along the four evaluation axes, measured against a Newton reference. Chapter 7 synthesizes those results, gives a verdict on each axis, and sets out the threats to validity. Chapter 8 closes with a summary and directions for future work.

Chapter 2

Background and theoretical foundations

This chapter establishes the mathematical foundations for the rest of the thesis. Its central object is the variational form of implicit Euler, which unifies XPBD and VBD as two approximate minimizers of the same energy function and therefore justifies a shared Newton solver as ground truth. The theoretical treatment follows Li et al. [5], which gives detailed descriptions and case studies of the theory and algorithms across most areas of physics-based simulation.

2.1 Time-stepping schemes for physical simulation

At its core, real-time physics reduces to advancing a state $(\mathbf{x}, \mathbf{v})$ — the positions and velocities of n particles — forward in time. Modern physics engines can be grouped into three families according to how they resolve constraints and compute motion: force-based, impulse-based, and position-based methods.

Force-based (Newtonian). Forces $\mathbf{f}(\mathbf{x}, \mathbf{v}, t)$ produce accelerations through Newton’s second law, $\mathbf{f} = \mathbf{M} \frac{d\mathbf{v}}{dt}$ with $\mathbf{v} = \frac{d\mathbf{x}}{dt}$, which rearranges to $\ddot{\mathbf{x}} = \mathbf{M}^{-1}\mathbf{f}$, where $\mathbf{M}$ is the mass matrix. This is the most intuitive approach. It is largely obsolete for real-time applications but is still used where high precision is required, such as engineering problems and offline visual effects.

Impulse-based. Velocities are manipulated directly through collision or constraint impulses. This family is popular for rigid bodies because it handles collisions as instantaneous changes in velocity. I do not pursue it here, since instantaneous velocity changes are not well suited for deformable bodies.

Position-based. Constraints are resolved by directly updating positions, traditionally bypassing force or acceleration steps. PBD and XPBD belong to this category. VBD is a hybrid: mathematically it is a position-based solver, since it directly updates positions, but it does so by solving a local optimization problem that mimics a force-based solver, so it is force-based in its physical formulation.

2.1.1 Explicit time integration (forward Euler method) and why it fails

The scheme is called explicit because it computes $\mathbf{x}_{t+h}$ and $\mathbf{v}_{t+h}$ directly, by substituting known values into simple formulas:

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h \mathbf{M}^{-1} \mathbf{f}(\mathbf{x}_t), \quad (2.1)$$

$$\mathbf{x}_{t+h} = \mathbf{x}_t + h \mathbf{v}_t. \quad (2.2)$$

Here $\mathbf{v}_t$ and $\mathbf{x}_t$ are the velocity and position at time t , and h is the time step. This method, however, is unstable [1]. More precisely, for spring systems, to keep stability, we have to preserve the spring's angular frequency $\omega = \sqrt{\frac{k}{m}}$, where k is the spring stiffness and m is the mass. For this we would need to choose a time step $h < \frac{2}{\omega} = 2\sqrt{\frac{m}{k}}$, which becomes impractically small for stiff materials, deeming it impractical for real-time cloth simulation.

2.1.2 Implicit time integration (backward Euler method)

Implicit time integration, in contrast, requires solving a system of equations to determine $\mathbf{x}_{t+h}$ and $\mathbf{v}_{t+h}$. The backward Euler scheme evaluates forces at the unknown end-of-step state:

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h \mathbf{M}^{-1} \mathbf{f}(\mathbf{x}_{t+h}), \quad (2.3)$$

$$\mathbf{x}_{t+h} = \mathbf{x}_t + h \mathbf{v}_{t+h}. \quad (2.4)$$

Now $\mathbf{v}_{t+h}$ depends on $\mathbf{x}_{t+h}$, and $\mathbf{x}_{t+h}$ depends on $\mathbf{v}_{t+h}$. This circular dependency is resolved by eliminating the velocity: substituting the velocity update into the position update gives a single equation, $\mathbf{x}_{t+h} = \mathbf{x}_t + h \mathbf{v}_t + h^2 \mathbf{M}^{-1} \mathbf{f}(\mathbf{x}_{t+h})$, in the unknown $\mathbf{x}_{t+h}$ alone; the velocity is then recovered as $\mathbf{v}_{t+h} = (\mathbf{x}_{t+h} - \mathbf{x}_t)/h$. The scheme is unconditionally stable for linear systems, but it may fail to converge if the initial approximation is far from the true solution, and it carries a real cost: a nonlinear root-finding problem must be solved at every step.

2.2 Implicit Euler as energy minimization

Effectively, solving the nonlinear system of equations in implicit Euler is the same as solving an optimization problem. This form is known as the variational form (also called the minimization or optimization form) of implicit Euler:

$$\begin{aligned} \mathbf{x}_{t+h} &= \underset{\mathbf{x}}{\operatorname{argmin}} G(\mathbf{x}), \\ G(\mathbf{x}) &= \frac{1}{2h^2} \|\mathbf{x} - \mathbf{y}\|_{\mathbf{M}}^2 + E(\mathbf{x}). \end{aligned} \quad (2.5)$$

where

$$\mathbf{y} = \mathbf{x}_t + h \mathbf{v}_t + h^2 \mathbf{M}^{-1} \mathbf{f}_{\text{ext}}.$$

Here $\mathbf{y}$ is the *inertial target* (the position reached under external forces alone), $\|\cdot\|_{\mathbf{M}}$ is the mass-weighted length, and $E(\mathbf{x})$ is the total potential energy evaluated at $\mathbf{x}$.

Equation (2.5) is central to the thesis, because **both XPBD and VBD are approximate minimizers of the same $G(\mathbf{x})$** . They differ only in *which unknowns* they optimize over. VBD minimizes G directly over the positions $\mathbf{x}$ — the *primal* variables — by

block-coordinate descent. XPBD instead rewrites the elastic energy as a set of compliant constraints and solves for the Lagrange multipliers (the constraint forces) that enforce them — the *dual* variables. This is precisely what makes one a primal and the other a dual method; Section 2.5 develops the distinction and what it predicts. Because both target the same minimizer, a Newton solve of (2.5) can serve as an unambiguous ground truth in Chapter 6.

2.3 Mass-spring systems

I use a mass-spring discretization of cloth: vertices carry point masses, and the edges of the cloth mesh carry springs. For a spring between particles i and j with rest length ℓ_0 and stiffness k , the constraint function and the associated energy are

$$C(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0, \quad (2.6)$$

$$E(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{2} k C^2. \quad (2.7)$$

The exact mass-spring structure used to represent cloth is introduced in Section 5.3.1.

2.3.1 Limitations of mass-spring models

The mass-spring model is cheap and easy to differentiate, but it is well known to be *simulation-mesh-dependent*: effective material parameters, such as macroscopic stiffness, depend on the discretization. Conclusions about absolute material parameters therefore do not transfer to FEM-based models. Conclusions about *relative* solver behavior — my actual subject — do transfer.

2.4 Solver structure: Gauss–Seidel vs. Jacobi updates

Both XPBD’s constraint sweep and VBD’s vertex-block sweep are Gauss–Seidel-style iterations: each local update uses the most recent neighbor values rather than the start-of-iteration values. A Jacobi variant, by contrast, updates all blocks in parallel from the start-of-iteration state; it converges more slowly in serial execution but maps better to the GPU.

A consequence, revisited in Section 7.3, is that both solvers are **order-dependent**. The constraint-traversal order (XPBD) and the vertex-traversal order (VBD) both affect the convergence rate and, in some cases, the converged solution itself. My experiments fix a deterministic order for reproducibility; I do not use randomization, since none of the canonical solver implementations do.

2.5 The primal/dual distinction

The unifying framing of this thesis is the primal/dual perspective of Macklin et al. [7]. Minimizing $G(\mathbf{x})$ in (2.5) can be approached in two ways.

Primal. The energy is minimized directly over the positions $\mathbf{x}$. A vertex-by-vertex block-coordinate descent on G is exactly VBD.

Dual. The energy is reformulated through compliant constraints — each spring becomes a soft constraint with compliance $\alpha = 1/k$ — and the solver works for the Lagrange multipliers $\boldsymbol{\lambda}$ on those constraints. Introducing the multipliers turns the pure minimization over $\mathbf{x}$ into a coupled problem in both $\mathbf{x}$ and $\boldsymbol{\lambda}$: it’s a *saddle-point* because its solution minimizes the energy in the position variables while maximizing it in the multipliers, rather than minimizing in all unknowns at once. XPBD solves this system with a Gauss–Seidel sweep, updating one constraint’s multiplier — and the positions it touches — at a time.

This duality is the thesis’s motivation for choosing the stress tests of Section 6.2.1 and Section 6.2.2, because it also reveals where the two approaches naturally differ. Dual variables couple constraints that share a particle, so a mass disparity at a shared particle is a scenario in which XPBD’s per-constraint updates may propagate slowly. The primal Hessian block at a vertex, on the other hand, collects all spring stiffnesses meeting there, so widely varying stiffnesses produce an ill-conditioned block — a scenario in which VBD’s per-vertex update may struggle.

Chapter 3

Related work

This chapter introduces the position-based and primal-descent lineages whose endpoints — XPBD and VBD — this thesis compares, and positions my work relative to the closest prior comparison.

3.1 From PBD to XPBD

Position-Based Dynamics (PBD), introduced by Müller et al. [11], targets real-time cloth and soft-body simulation in games. Each iteration projects the predicted positions onto the constraint set — moving the particles each constraint couples until the constraint is satisfied — through Gauss–Seidel sweeps, and recovers velocities by finite difference. PBD is unconditionally stable, but it has a well-known flaw: *constraint stiffness depends on the iteration count and the time step*.

XPBD, introduced by Macklin et al. [8], fixes this by augmenting each constraint with a compliance term $\alpha = 1/k$ and solving for the associated Lagrange multiplier λ . The resulting iteration solves an implicit-Euler problem with arbitrary elastic energies, and constraint stiffness becomes a true material parameter, independent of the iteration count. As Ma et al. [6] summarize the lineage:

“Early research focused on force–object relationships, leading to the development of models such as the Mass-Spring System, Finite Element Method, and Rigid Body Dynamics. However, these models have limitations, including performance degradation, lack of real-time capability, and unrealistic motion as simulation complexity increases. PBD was introduced to address these limitations. Initially developed by Jakobsen, PBD provides a stable and efficient approach by applying wedge-based constraints instead of force-based methods. Macklin et al. further refined and optimized PBD for real-time interaction.”

Small steps in physics simulation. Macklin et al. [9] (the “Small Steps” note) later argued that XPBD converges most reliably with *many short substeps and a single iteration per substep*, rather than a single long step with many iterations. With only one iteration per substep the multiplier term entering the update is always zero — λ is reset each substep and never re-used across iterations — so there is no need to store and carry it between iterations. This version is now a standard XPBD configuration, but for the sake of a fair comparison — to compare methods across substep and iteration counts — I deliberately leave this optimization out of the core comparison.

3.2 Vertex Block Descent (VBD)

Vertex Block Descent, due to Chen et al. [3], treats each vertex as a tiny optimization block: at each iteration, every vertex moves to the minimizer of G with all other vertices held fixed. This requires the local 3×3 Hessian of the energies sharing the vertex and a Newton step on that block. Ma et al. [6] place the method as follows:

“More recently, the VBD algorithm that has been proposed provides an efficient approach to parallelize large-scale computations. By processing nodes in block units, VBD significantly enhances performance, particularly in simulations involving a high number of nodes.”

The original VBD paper contributes three features beyond the basic block descent. *Chebyshev acceleration* is an over-relaxation that significantly speeds convergence, at the cost of the method’s energy guarantees. *Adaptive initialization* seeds each step’s initial guess from a blend of the full inertial prediction and an inertia-only extrapolation, leaning on the latter for vertices near rest to avoid over-shooting under gravity; the exact blend formula is given in Section 4.2.3. *Graph-coloring parallelism* lets vertices that share no constraint be updated independently, enabling massive parallelism. The fairness rationale for enabling adaptive initialization while disabling Chebyshev in the core comparison is given in Section 5.4.4.

A follow-up, Augmented VBD (AVBD) [4], improves stiffness-ratio handling, infinite-stiffness handling, and convergence behavior. To keep the baseline comparison anchored to the established formulation, I restrict the implementation to the canonical 2024 VBD; I cite AVBD in Section 6.4 as the most relevant counter-evidence on the stiffness-ratio axis, and revisit the question it raises in Section 7.3.4.

3.3 Prior comparison: Ma et al. (2024)

The most directly comparable prior work is Ma et al. [6], who compare cloth simulation under PBD and VBD in Unity and establish that VBD scales better than PBD with node count. My work differs in four substantial ways:

1. **XPBD, not PBD.** Plain PBD has the stiffness-coupling flaw that XPBD was designed to fix, so a PBD-vs-VBD comparison conflates algorithmic differences with this known bug. Using XPBD isolates the primal/dual distinction.
2. **Canonical VBD.** Ma et al.’s pseudocode departs from [3] in two concrete ways relevant to the comparison: it does not include the adaptive-initialization blend (Section 4.2.3), which always starts each step from the full inertial target; and it does not include Chebyshev acceleration. Both omissions affect convergence on stiff problems, where the canonical warm start and over-relaxation carry most of VBD’s reported per-step speedup. I follow canonical VBD, but disable Chebyshev for the core comparison so I read off algorithmic properties rather than acceleration tricks; adaptive initialization is kept enabled because it is VBD’s inertial prediction and has a direct counterpart in XPBD’s explicit guess. The fairness rationale is detailed in Section 5.4.4.
3. **Beyond frame rate.** Ma et al. report fps and node counts. I add a Newton ground truth, a solution quality vs. computational cost comparison, and targeted stress tests (mass ratio, stiffness ratio, large dt) keyed to the primal/dual distinction.

4. **Varying the substep count.** Their comparison kept the substep count fixed throughout, so it did not consider how changing it affects each algorithm.

In short, this thesis answers a question that Ma et al. explicitly left as future work: how does VBD compare to *XPBD*, and at *matched quality* rather than only at matched node counts?

3.4 Reference materials

The Physics-Based Simulation book of Li et al. [5] provided canonical derivations and pedagogical notes; its treatment of implicit Euler as variational minimization directly informs the framing in Section 2.2. The *Ten Minute Physics* tutorial series [10] was used as a sanity-check reference for the correctness of the XPBD implementation, and the TinyVBD reference implementation [2] served the same role for the VBD side.

Chapter 4

The two algorithms

This chapter states a simplified form of the XPBD and VBD algorithms I implemented in Chapter 5. Both target the same variational problem (2.5), but they reach it through opposite descent directions.

4.1 XPBD: dual-variable position projection

4.1.1 Derivation

For each elastic spring, XPBD [8] associates a constraint $C(\mathbf{x}) = 0$ at zero deformation with a compliance $\alpha = 1/k$, the inverse of the spring stiffness. Because the implicit time stepping (Section 2.1.2) carries a factor of $1/h^2$ on the inertial term of (2.5), the compliance enters the update only after being scaled by the squared substep size. Folding that factor into the compliance defines the *time-step-scaled compliance* $\tilde{\alpha} = \alpha/h^2$, which is what lets XPBD reproduce an effective stiffness that is independent of h and of the iteration count. The quantity $\lambda = -\tilde{\alpha}^{-1}C(\mathbf{x})$ is the Lagrange multiplier for that constraint and can be interpreted as the magnitude of the spring force. The augmented Lagrangian of (2.5) with a single substep h has the stationarity condition

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) - \nabla \mathbf{C}(\mathbf{x})^T \boldsymbol{\lambda} = \mathbf{0}, \quad (4.1)$$

$$\mathbf{C}(\mathbf{x}) + \tilde{\alpha} \boldsymbol{\lambda} = \mathbf{0}, \quad (4.2)$$

where $\mathbf{C}$ and $\boldsymbol{\lambda}$ stack the per-constraint functions and multipliers. XPBD solves this coupled system with a Gauss–Seidel sweep over the constraints: for each constraint j , the multiplier increment is

$$\Delta \lambda_j = \frac{-C_j(\mathbf{x}) - \tilde{\alpha}_j \lambda_j}{\nabla C_j \mathbf{M}^{-1} \nabla C_j^T + \tilde{\alpha}_j}, \quad (4.3)$$

and the position correction is

$$\Delta \mathbf{x} = \mathbf{M}^{-1} \nabla C_j^T \Delta \lambda_j. \quad (4.4)$$

Velocities are recovered at the end of each substep by $\mathbf{v}_{t+h} = (\mathbf{x}_{t+h} - \mathbf{x}_t)/h$.

4.1.2 Algorithm

Algorithm 1 shows the simplified form of XPBD used for the core comparison: no damping, no collision handling. The full implemented version is given in Section A.2.1 with a more detailed pseudocode.

Algorithm 1 XPBD step (core comparison). n_{sub} substeps, each with n_{it} Gauss–Seidel iterations over the constraints.

```

1:  $h \leftarrow dt/n_{\text{sub}}$  ▷ substep size
2: for  $s = 1$  to  $n_{\text{sub}}$  do
3:   for all vertices  $i$  do
4:      $\mathbf{v}_i \leftarrow \mathbf{v}_i + h(\mathbf{g} + w_i \mathbf{f}_{\text{ext}}(\mathbf{x}_i))$  ▷  $w_i = 1/m_i$ 
5:      $\mathbf{x}_i^{\text{prev}} \leftarrow \mathbf{x}_i$ 
6:      $\mathbf{x}_i \leftarrow \mathbf{x}_i + h \mathbf{v}_i$  ▷ inertial target  $\mathbf{y}_i$ 
7:   end for
8:   for all constraints  $j$  do
9:      $\lambda_j \leftarrow 0$  ▷ multipliers reset every substep
10:  end for
11:  for  $\text{it} = 1$  to  $n_{\text{it}}$  do
12:    PROJECTCONSTRAINTS( $C_1, \dots, C_M, \mathbf{x}, \boldsymbol{\lambda}$ )
13:  end for
14:  for all vertices  $i$  do
15:     $\mathbf{v}_i \leftarrow (\mathbf{x}_i - \mathbf{x}_i^{\text{prev}})/h$ 
16:  end for
17: end for

18: procedure PROJECTCONSTRAINTS( $C_1, \dots, C_M, \mathbf{x}, \boldsymbol{\lambda}$ )
19:   for all constraints  $j$  do ▷ one Gauss–Seidel sweep
20:      $\tilde{\alpha}_j \leftarrow \alpha_j/h^2$ 
21:      $\Delta\lambda_j \leftarrow \frac{-C_j(\mathbf{x}) - \tilde{\alpha}_j \lambda_j}{\nabla C_j \mathbf{M}^{-1} \nabla C_j^T + \tilde{\alpha}_j}$ 
22:      $\lambda_j \leftarrow \lambda_j + \Delta\lambda_j$ 
23:      $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{M}^{-1} \nabla C_j^T \Delta\lambda_j$ 
24:   end for
25: end procedure

```

4.1.3 Characteristics

XPBD uses *dual variables*: an iteration’s state is encoded in the multipliers $\boldsymbol{\lambda}$, and positions are derived from them. It handles *stiffness through compliance*, since material stiffness enters only via $\tilde{\alpha} = 1/(kh^2)$, so very stiff constraints naturally degenerate to hard projection ($\tilde{\alpha} \rightarrow 0$). It uses *first-order gradient information only*: each constraint contributes its gradient ∇C_j , and no Hessian of E is built. It *prefers substeps over iterations*: its widely used small-steps optimization strategy only uses one iteration but performs many substeps. As a member of the position-based family it is also *highly stable and computationally cheap*: it never evaluates forces, working directly on positions through constraint projection instead, which avoids the stiffness-driven instabilities of explicit force integration and keeps the per-step cost low.

4.2 VBD: primal block-coordinate descent

4.2.1 Derivation

VBD [3] performs block-coordinate descent on G in (2.5), with one block per vertex. Holding all other vertices fixed, the update for vertex i is the Newton step on the local 3-vector problem

$$\Delta \mathbf{x}_i = -\mathbf{H}_i^{-1} \mathbf{g}_i, \quad (4.5)$$

where

$$\mathbf{g}_i = \frac{m_i}{h^2} (\mathbf{x}_i - \mathbf{y}_i) + \sum_{e \in \mathcal{F}_i} \frac{\partial E_e(\mathbf{x})}{\partial \mathbf{x}_i}, \quad (4.6)$$

$$\mathbf{H}_i = \frac{m_i}{h^2} \mathbf{I}_3 + \sum_{e \in \mathcal{F}_i} \frac{\partial^2 E_e(\mathbf{x})}{\partial \mathbf{x}_i \partial \mathbf{x}_i}. \quad (4.7)$$

Here $\mathbf{g}_i$ is the gradient of the local variational energy around vertex i , $\mathcal{F}_i$ is the set of force elements acting on vertex i , and $\mathbf{H}_i$ is the local Hessian. For a distance-spring energy $E_e = \frac{1}{2}k(\|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0)^2$, the per-vertex gradient and Hessian are closed-form and inexpensive to evaluate.

4.2.2 Algorithm

As with XPBD, the form given in Algorithm 2 below is the simplified, core-comparison version: no damping, no collisions, no Chebyshev acceleration. Adaptive initialization is kept on (defined in Section 4.2.3); the fairness rationale for both choices is given in Section 5.4.4. Here I use the fixed-iteration-count variant in place of the convergence-based early-stopping one. The full implemented version — with damping, self-collision, and Chebyshev acceleration — is given in Section A.2.2 and uses the early-stopping variant.

4.2.3 Adaptive initialization

The starting guess on line 5 of Algorithm 2, taken from Chen et al. [3], blends two extremes per vertex: the full inertial target $\mathbf{x}_i + h\mathbf{v}_i + h^2\mathbf{a}_i^{\text{ext}}$ ($\tilde{a}_i = 1$) and the pure velocity extrapolation $\mathbf{x}_i + h\mathbf{v}_i$ ($\tilde{a}_i = 0$). The blend factor is read off the previous step’s acceleration $\mathbf{a}_i^{\text{prev}} = (\mathbf{v}_i^t - \mathbf{v}_i^{t-h})/h$ as

$$\tilde{a}_i = \text{clamp}_{[0,1]} \left(\frac{\mathbf{a}_i^{\text{prev}} \cdot \mathbf{a}_i^{\text{ext}}}{\|\mathbf{a}_i^{\text{ext}}\|^2} \right), \quad (4.8)$$

which is the projection of the realized acceleration onto the external one. $\tilde{a}_i \approx 1$ identifies a vertex that was in free fall last step, so the inertial target is a good warm start. $\tilde{a}_i \approx 0$ identifies a vertex that was held in place by stiff springs or a pin, where the inertial target sits far from the minimizer of G and the block-descent sweep would otherwise spend iterations dragging it back; starting close to the previous position is the better option there. XPBD has no analogous adaptive blend — it always starts from the full inertial target ($\tilde{a} = 1$) and lets the constraint sweeps do the rest. The reasoning for keeping adaptive initialization enabled, and its acknowledged bias toward VBD, is discussed in Section 5.4.4.

Algorithm 2 VBD step (core comparison). n_{sub} substeps, each with n_{it} block-descent iterations over the vertices.

```

1:  $h \leftarrow dt/n_{\text{sub}}$ 
2: for  $s = 1$  to  $n_{\text{sub}}$  do
3:   for all vertices  $i$  do
4:      $\mathbf{a}_i^{\text{ext}} \leftarrow \mathbf{g} + w_i \mathbf{f}_{\text{ext}}(\mathbf{x}_i)$ 
5:      $\mathbf{y}_i \leftarrow \mathbf{x}_i + h \mathbf{v}_i + h^2 \mathbf{a}_i^{\text{ext}}$  ▷ inertial target
6:      $\mathbf{x}_i^{\text{prev}} \leftarrow \mathbf{x}_i$ 
7:      $\mathbf{x}_i \leftarrow \mathbf{x}_i + h \mathbf{v}_i + \tilde{a}_i h^2 \mathbf{a}_i^{\text{ext}}$  ▷ adaptive initial guess,  $\tilde{a}_i \in [0, 1]$ 
8:   end for
9:   for  $\text{it} = 1$  to  $n_{\text{it}}$  do
10:    SWEEP( $\mathbf{x}, \mathbf{y}$ ) ▷ one Gauss–Seidel pass over vertices
11:   end for
12:   for all vertices  $i$  do
13:     $\mathbf{v}_i \leftarrow (\mathbf{x}_i - \mathbf{x}_i^{\text{prev}})/h$ 
14:   end for
15: end for

16: procedure SWEEP( $\mathbf{x}, \mathbf{y}$ )
17:   for all vertices  $i$  do
18:     $\mathbf{g}_i \leftarrow \frac{m_i}{h^2}(\mathbf{x}_i - \mathbf{y}_i) + \sum_{e \in \mathcal{F}_i} \frac{\partial E_e(\mathbf{x})}{\partial \mathbf{x}_i}$  ▷ gradient ( $= -\mathbf{f}_i$ )
19:     $\mathbf{H}_i \leftarrow \frac{m_i}{h^2} \mathbf{I}_3 + \sum_{e \in \mathcal{F}_i} \frac{\partial^2 E_e(\mathbf{x})}{\partial \mathbf{x}_i \partial \mathbf{x}_i}$ 
20:     $\mathbf{x}_i \leftarrow \mathbf{x}_i - \mathbf{H}_i^{-1} \mathbf{g}_i$  ▷ local  $3 \times 3$  Newton step
21:   end for
22: end procedure

```

4.2.4 Characteristics

VBD is *primal*: an iteration’s state is the positions $\mathbf{x}$, with no extra multipliers. It is *second-order locally*, since each block update is a Newton step on the true elastic Hessian restricted to one vertex. It *prefers iterations over substeps*: VBD is normally used [3] with one long step subdivided into many block iterations, and the implicit Hessian handles stiffness even at larger h , thus being *stable even under large time steps*, which is especially valuable when computational resources are constrained. Finally, it is *parallel-friendly*: vertices that share no constraint can be updated independently through graph coloring, which, when parallelized, makes it particularly well suited to large-scale simulations.

4.3 Side-by-side: where the algorithms differ

The first and third rows capture the distinction that the primal/dual lens (Section 1.3) builds on, and the one the experiments return to when interpreting each result.

	XPBD	VBD
Variable updated	multipliers λ	positions $\mathbf{x}$
Order of information	gradient ∇C	gradient + Hessian of E
Block size	per-constraint	per-vertex
Stiffness handled by	compliance $\tilde{a}$	implicit Hessian
Typical configuration	many substeps, 1 iteration	one step, many iterations
Parallelism strategy	constraint coloring (harder)	vertex coloring (easier)

Table 4.1: Algorithmic contrast between XPBD and VBD.

Chapter 5

Methodology and implementation

This chapter describes the implementation that produces the data in Chapter 6 and, more importantly, the decisions that keep the XPBD-vs-VBD comparison fair.

5.1 The Unity game engine

A game engine is an integrated software framework that bundles the building blocks a real-time interactive application needs — rendering, asset and scene management, input handling, a scripting layer, and usually a physics module — behind a single editor and a consistent programming interface. Luckily, this doesn't just make it good for games, VR and AR uses, but also for simulations.

Unity is one of the most popular game engines. It is cross-platform, scripted in C#, and organized around a *component model*: behavior is attached to objects in the scene as self-contained components, each of which can expose parameters in the editor and hook into the engine's update loop through callbacks. A simulator, in this model, is simply a component attached to a cloth object; references to other objects in the scene (a collider, a target transform) are assigned by dragging them onto the component in the editor, with no extra wiring code.

Unity makes it easy to set up experiments, as most of the time it's enough to create a new scene, drag and drop objects into it (in my case, cloth models were simple plane meshes imported from Blender), and assign the relevant components to them, which then determine what solver runs. It also has several built-in tools that proved useful for data collection: the Profiler for gathering FPS statistics, the Recorder for capturing videos of the experiments, and Shader Graph for creating textures in a straightforward way.

5.1.1 Pinning vertices through Unity components

Several experiments require parts of the cloth to be held in place — for example, pinning the top edge of a hanging cloth. I implement this boundary condition with small, separate Unity components rather than inside the solver, so that the pinning logic is independent of which solver is running. Each such script is attached to the cloth object, and the external object that the cloth should follow — shown in gray in the experiment figures — is supplied to the script by assigning its collider in the editor.

The script identifies the cloth vertices that lie on the assigned object and fixes them in two steps. It sets the *inverse mass* of each of those vertices to zero, which makes them

immovable as far as the solver is concerned: a vertex with zero inverse mass has effectively infinite mass and is never displaced by the constraint solve. It then overwrites the positions of those same vertices in its **Update** method so that they track the external object every frame. Each gray object is followed by the vertices attached to it; for the rectangle that pins the upper edge of the cloth, the followed vertices also inherit the object’s rotation and scale, so moving, turning, or stretching that object drags the pinned edge with it.

5.2 Platform and environment

The simulators are implemented inside the Unity Editor. All measurements reported in Chapter 6 were collected in the following environment:

- **CPU:** AMD Ryzen 7 5800H
- **Memory:** 16 GB DDR4
- **GPU:** NVIDIA GeForce RTX 3060 (Laptop)
- **Operating system:** Windows 11 Pro
- **Development tool:** Unity 6000.4.0f1

5.3 Integration architecture

Both solvers share a common scene scaffolding, and the simulations are driven by the **Update** method of Unity’s **MonoBehaviour** classes. The scaffolding handles mesh and spring-topology construction at scene load, the recording of statistics such as frame time and memory, and rendering, which runs on a separate thread from the simulator.

In the Unity Editor, the **Update** method is called as often as the environment allows; if a step takes more compute time than the frame time, the next frame is delayed until it finishes. The **Update** method therefore effectively provides a variable frame rate that can be used to measure computation time. To keep the experiments deterministic and easier to control, I use a fixed simulation time step of $dt = 1/30$ s. This is also a relatively large time step, which makes it a good test of the solvers’ stability and lets the substep count carry more meaning in Section 6.1.1.

Overhead of the shared scaffolding. The scaffolding adds a small, practically constant per-frame overhead, mostly from Unity-specific calls and rendering. I measure FPS with the Unity Profiler, which counts the actual frames drawn, so the reported value includes this overhead; but because both solvers pay it identically, it does not bias the comparison. To keep performance as high as possible, I favor primitive types, arrays, and structs over classes and lists, and I cache frequently accessed variables globally to shift the load from the CPU to memory.

5.3.1 Cloth model construction

The mass-spring cloth model I implement is the one used by Müller [10]. Each internal vertex is connected to its neighbors by three types of distance constraint, each with its own stiffness/compliance value. **Stretching** constraints connect a vertex to its immediate

horizontal and vertical neighbors; **shear** constraints connect it to its diagonal neighbors; and **bending** constraints connect it to its second horizontal and vertical neighbors. An internal vertex therefore has twelve constraints attached to it. Note that the bending constraints are not dihedral-angle constraints: I deliberately use a single constraint form to keep the comparison less complex.

The mass of each vertex is set proportionally to the area of cloth it represents: a vertex receives a quarter of the summed area of the quadrilaterals it touches, and the parameter the user supplies is the total mass of the cloth. This keeps the cloth’s behavior consistent across resolutions. Assigning the same mass to every vertex instead would also leave the side and corner vertices effectively heavier relative to the area they represent.

5.4 How the comparison is kept fair

I adopt several explicit fairness controls, described below.

5.4.1 Stiffness $\leftrightarrow$ compliance mapping

VBD takes a stiffness k directly, whereas XPBD takes a compliance $\alpha = 1/k$. To match materials, I set

$$\alpha_{\text{XPBD}} = 1/k_{\text{VBD}}. \quad (5.1)$$

This mapping is verified in Section 6.3.1: a single spring with known analytic parameters yields the same oscillation frequency and decay rate in both solvers.

5.4.2 Matched compute budget — and the substep/iteration caveat

The natural unit of work differs between the two methods: XPBD’s primary lever is the substep count, while VBD’s is the inner iteration count. **A substep is not an iteration.** A substep recomputes the inertial target $\mathbf{y}$ and solves a fresh constraint system, whereas an iteration refines the same step. Because of this asymmetry, I compare the two methods on the basis of *wall-clock time* throughout Chapter 6. One source of bias is that VBD uses a maximum iteration count while XPBD uses a fixed iteration count. Either method could implement either scheme, but these choices are inherent to each method’s design (even allowing for their future optimizations), so I keep them as they are.

5.4.3 Matched gravity, dt , and damping

Unless otherwise noted, all experiments use gravity $\mathbf{g} = (0, -9.81, 0)$ m/s² and a simulation time step of $dt = 1/30$ s. Rayleigh damping is disabled in both methods, because the difference in solver structure produces different results from the same damping parameters.

5.4.4 Handling VBD’s accelerations for the core comparison

The 2024 VBD paper [3] reports significant speedups from Chebyshev acceleration and adaptive initialization. Adaptive initialization is VBD’s own inertial prediction, which is also present in XPBD (which guesses with explicit Euler), so I keep it enabled. Chebyshev acceleration is likewise part of VBD, but it confounds an apples-to-apples comparison, since

XPBD could implement the same technique, as shown in [13]; I therefore disable it for the core comparison.

5.4.5 Hessian projection for stability

The original VBD paper [3] argues that the Hessian need not be positive-definite: even if a step occasionally does not point in a descent direction, the small number of such steps does not affect overall convergence. In my experiments, however, projecting each local Hessian to positive-definite let the method handle more extreme scenarios without noticeably changing its behavior or performance, so I keep the projection on throughout.

5.5 Self-collision handling

The two solvers implement self-collision differently, by necessity. *XPBD self-collision* uses non-penetration constraints projected each iteration — contact is enforced as a position constraint. *VBD self-collision*, in contrast, uses an elastic penalty potential added to E , contributing a gradient and Hessian to the affected vertex blocks.

This is a model difference, not a solver difference: a penalty model *could* be implemented in XPBD and a projection model *could* be implemented in VBD, but the natural, most solver-fitting form differs between them. The self-collision experiment in Section 6.2.3 is therefore purely qualitative.

For both solvers, I used the implementation shown in [10]. It uses a spatial hash in the broad phase to decrease the number of vertex pairs to check. As it works with discrete collision detection, it caps the velocity, so that fast-moving vertices cannot tunnel through each other.

5.6 Parallelization (forward-looking)

All measurements in this thesis are run single-threaded on the CPU. As noted here and revisited in Section 7.3, the two solvers have different parallelization profiles. *VBD* parallelizes naturally over vertex blocks through graph coloring: vertices that share no constraint can be updated independently within a color class, which suits GPU execution well. *XPBD* can be parallelized through edge coloring, but this coloring is finer-grained (per constraint — of which there are usually many more than vertices — rather than per vertex), so it produces more color classes and hence more sequential dependencies.

The expected result — that parallelization favors VBD more than XPBD — is plausible but unverified by my experiments. It stands as a threat to the external validity of any claim about which solver is “faster overall.”

Chapter 6

Experiments and results

This chapter reports measurements along the four evaluation axes introduced in Section 1.4: efficiency, robustness, physical realism, and flexibility. Unless stated otherwise, every experiment uses the shared scaffolding and fairness controls of Section 5.4: matched gravity, $dt = 1/30$ s, no damping, $\alpha_{\text{XPBD}} = 1/k_{\text{VBD}}$, and Chebyshev acceleration disabled in VBD for the core comparison.

6.1 Computational efficiency

6.1.1 Influence of substep and iteration count on error and compute time

For each solver I sweep over a range of substep and iteration configurations and plot the Pareto front of the resulting runs — the set of configurations not dominated by any other, meaning that no run is simultaneously faster and more accurate — using average ms/frame against the average root-mean-squared position error measured against a Newton reference. Both solvers are swept over every combination of the substep and iteration counts $n_{\text{sub}} \in \{1, 2, 4, 8, 16, 32\}$ and $n_{\text{it}} \in \{1, 2, 4, 8, 16, 32\}$. For this problem the iteration count is enforced within VBD (no early stopping), to keep the comparison clean. Each diagram in Figure 6.1 uses a Newton reference computed with a different number of substeps.

The test problem is a chain of ten nodes with one end pinned and a weight of 100 at the free end. The nodes are connected in sequence, one to the next, with no other constraints between them. Initially the free end is raised to the same height as the pinned end and positioned so that the chain is neither stretched nor compressed, and only gravity acts on the system.

My results show that a single VBD iteration always approximates the Newton solution more closely than a single XPBD iteration, but it is also more expensive. Especially against the lower-iteration references (1 and 4 substeps), VBD usually achieves better performance by increasing its iteration count, whereas XPBD’s best solutions instead come from increasing the number of substeps. This is clearest in the diagram with the 1-substep Newton reference, where several of XPBD’s best solutions come from raising the substep count rather than the iteration count. For harder problems — where the time step is large enough, with fewer substeps — VBD takes the lead. As the reference becomes more refined with more substeps, the sub-problems become easy enough for both solvers

to converge in one or two iterations, and at that point XPBD takes over thanks to its cheaper per-iteration cost.

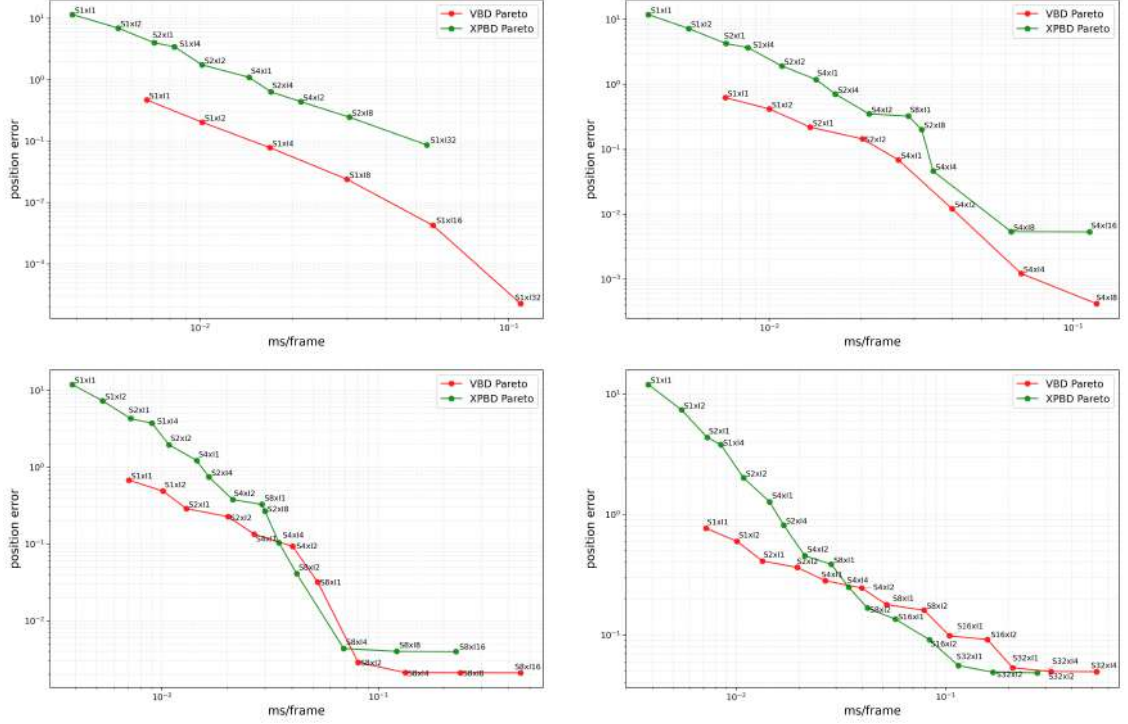


Figure 6.1: Pareto fronts of average ms/frame versus average RMS position error on the 10-node weighted chain. The Newton reference uses 1 substep (top-left), 4 substeps (top-right), 8 substeps (bottom-left), and 64 substeps (bottom-right).

6.1.2 Frame time vs. node count

To measure how each solver scales with problem size, I record the average frames per second on a hanging-cloth scene with wind interaction, at mesh resolutions from 8×8 to 128×128 , holding the solver settings fixed. The primary comparison is XPBD with 4 substeps and 4 iterations ($XPBD_{sub4 \times it4}$) against VBD with 4 substeps and a maximum of 4 iterations ($VBD_{sub4 \times it_{max}4}$), which gives the “same parameter” comparison. To see how much VBD gains from early stopping, I also include VBD variants with enforced iteration counts: $VBD_{sub4 \times it4}$, which shows how the two algorithms compare with truly matching parameters, and $VBD_{sub4 \times it2}$, which the Pareto fronts of Section 6.1.1 suggest performs close to $XPBD_{sub4 \times it4}$ in both “realism” and “efficiency.”

In my tests, $VBD_{sub4 \times it2}$ always outperforms $XPBD_{sub4 \times it4}$, while $VBD_{sub4 \times it4}$ always trails it slightly. As expected, $VBD_{sub4 \times it_{max}4}$ provides a fair middle ground, outperforming XPBD on the easier problems and converging toward its own enforced variant on the harder ones. Figure 6.2 shows a 32×32 -node cloth in this experiment’s test environment as it progresses in time.

Configuration	8×8	16×16	32×32	64×64	128×128
$XPBD_{sub4 \times it4}$	133.8	84.5	36.0	10.6	2.8
$VBD_{sub4 \times it_{max}4}$	138.1	91.2	31.8	8.3	2.1
$VBD_{sub4 \times it2}$	136.0	92.4	42.8	13.6	3.7
$VBD_{sub4 \times it4}$	124.7	73.4	29.2	8.1	2.1

Table 6.1: Steady-state average FPS on the wind-driven hanging cloth at increasing mesh resolutions.

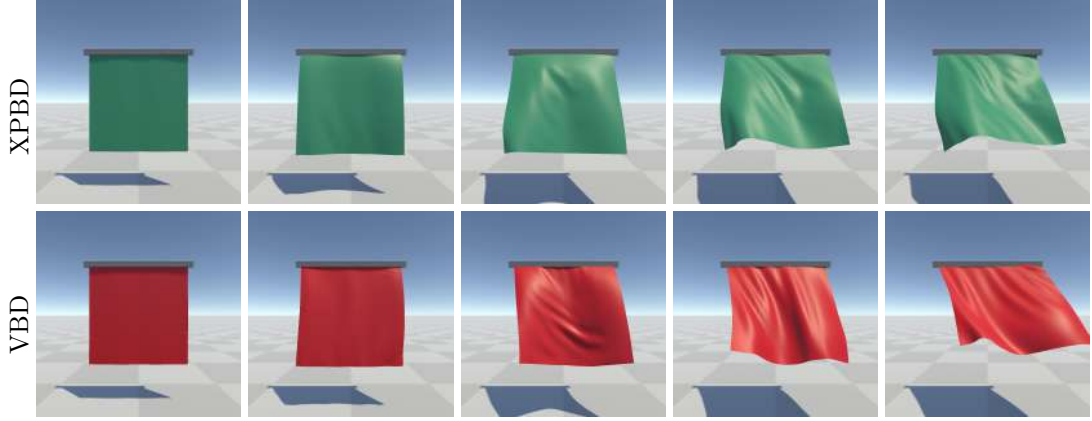


Figure 6.2: Wind-driven hanging cloth used in Table 6.1.

6.2 Robustness and stability

6.2.1 Chain with heavy weight attached

This scene is a chain fixed at one end, with the particle at the other end given a mass 1000 times greater than the others. Figure 6.3 compares XPBD and VBD against a reference produced by a Newton solver; all three methods use a single substep and 100 iterations. As predicted by the primal/dual paper [7], XPBD — a dual solver — fails to converge to the reference solution, whereas VBD approaches it far more closely.

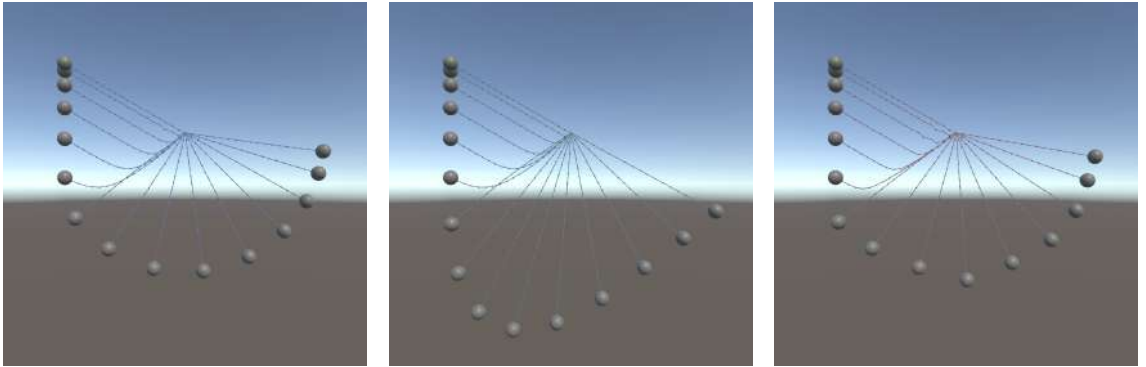


Figure 6.3: Chain with a $1000\times$ heavier node at the free end. Left to right: Newton, XPBD, VBD.

6.2.2 Chain with high stiffness ratio

This scene is a chain of alternating soft and stiff springs, with a stiffness ratio of 1:10000, mirroring the experiment in the original VBD paper [3]. As in the previous experiment, all three methods in Figure 6.4 use one substep and 100 iterations. The roles are now reversed: VBD — a primal solver — does not converge to the reference solution, while XPBD matches it almost perfectly.

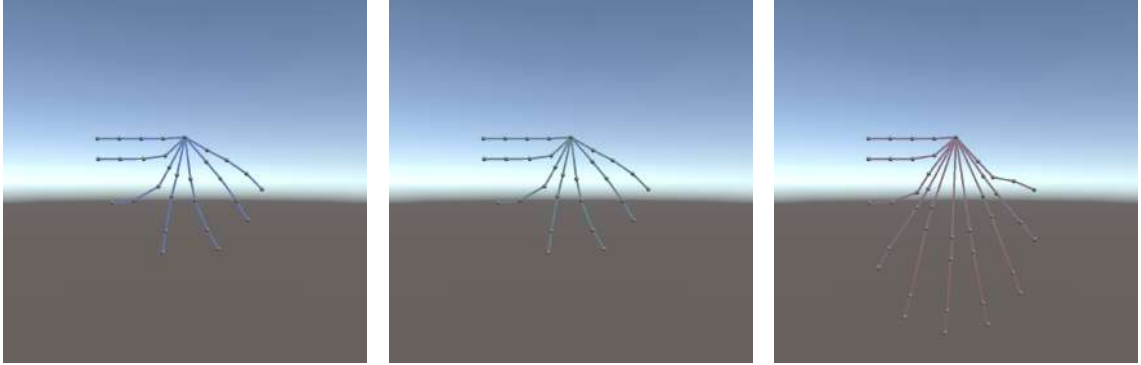


Figure 6.4: Chain with alternating springs at a 1:10000 stiffness ratio. Left to right: Newton reference, XPBD, VBD.

These two experiments also reinforce that my XPBD and VBD implementations are correct, since both behave exactly as the theory predicts. This raises confidence in the rest of my results and makes clearer that the behavior I report reflects the algorithms rather than bugs or implementation errors.

6.2.3 Self-collision: cloth folded onto itself

Both solvers can also be used to simulate collisions. In this scene I drop a cloth onto the ground so that it folds onto itself. As noted in Section 5.5, the two solvers implement self-collision differently, so rather than give exact measurements I report my observations.

Physically accurate cloth requires handling many stiff constraints: woven fabrics, for instance, stretch by only about 5–10%. The cloth model of Section 5.3.1 attaches twelve springs of three different stiffness values to each interior node, and those values can differ greatly — exactly the setup that the stiffness-ratio experiment above shows to be a serious disadvantage for VBD. The result is very unstable behavior: even when the simulation does not blow up, the cloth overshoots most configurations. Initial vertex noise on the scale of 0.001 produces large movements — visible in the first image, where the cloth changes shape even before colliding with the ground — and the cloth keeps shifting along the floor even after it appears to have settled.

The instability can be tamed by giving the vertices a large enough mass; my experiments suggest the vertex mass needs to be at minimum within two to three orders of magnitude of the largest stiffness value. This is exactly why the original VBD paper [3] uses a different cloth model, and it shows that VBD sometimes has to be hand-tuned to stay stable. VBD also cannot handle infinitely stiff constraints. AVBD [4] improves substantially on this axis and is more robust, but between canonical VBD and XPBD, XPBD is the more robust solver for this problem.

Figure 6.5 and Figure 6.6 show the drop at a total cloth mass of 1 and of 10000, respectively; in each, the top row is XPBD and the bottom row is VBD.

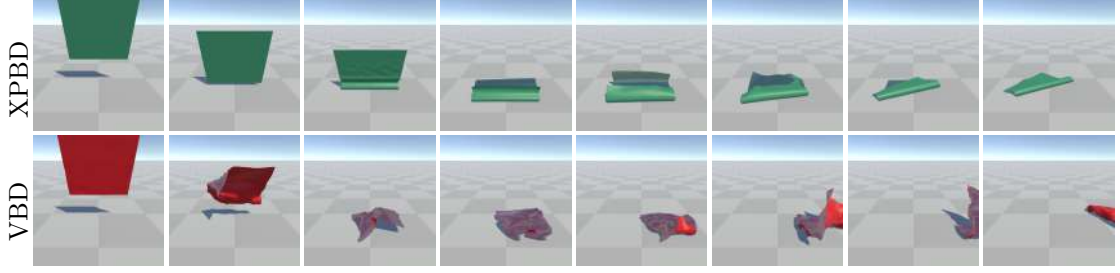


Figure 6.5: Cloth drop and self-collision at total mass 1, over time (left to right). Top: XPBD. Bottom: VBD.

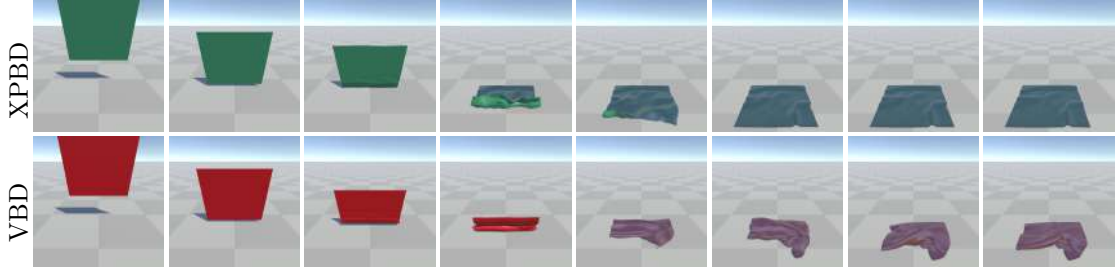


Figure 6.6: The same cloth drop at total mass 10000, over time (left to right). Top: XPBD. Bottom: VBD. The larger vertex mass stabilizes the VBD solve.

6.3 Physical realism

6.3.1 Harmonic oscillator

The harmonic oscillator is implemented as a single distance constraint between a pinned top node and a free bottom node, and I measure its amplitude over time. To compare against Fig. 2 of the XPBD paper [8], I use the same parameters, only with the direction reversed: rest length 1, compliance $\alpha = 10^{-3}$, rest position $x = -1$, initial position $x_0 = -0.5$, and mass $m = 1$.

Regardless of dt , both solvers converge in a single iteration and follow the same reference curve perfectly. Figure 6.7 also shows the analytic solution $x(t) = x_0 \cos(\omega t)$ with $\omega = \sqrt{k/m} = 31.62 \text{ s}^{-1}$ as dashed lines. As in the XPBD example, all of the visible damping is a side effect of the implicit discretization of the equations of motion.

6.3.2 Energy tracking

Here I plot the total mechanical energy over time for an undamped 16×16 cloth released from 20 degrees, in order to identify systematic energy gain (instability) or loss (numerical damping) for each solver. I use VBD with 64 maximum iterations as the reference solution. Each test is conducted without substepping.

Over a longer time frame, Figure 6.8 shows that both solvers lose energy along the same exponential decay curve. The oscillations around that curve are purely a convergence artifact: the solver over- and undershoots because of its gradient estimate.

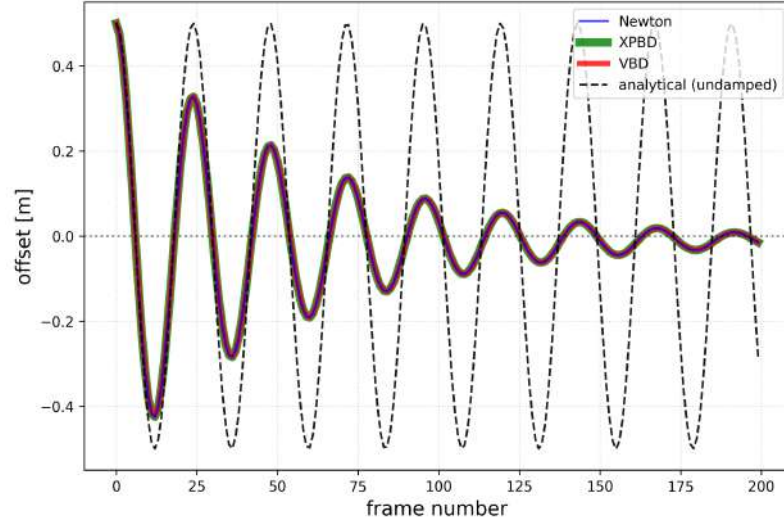


Figure 6.7: Harmonic-oscillator amplitude over time for both solvers against the analytic solution (dashed).

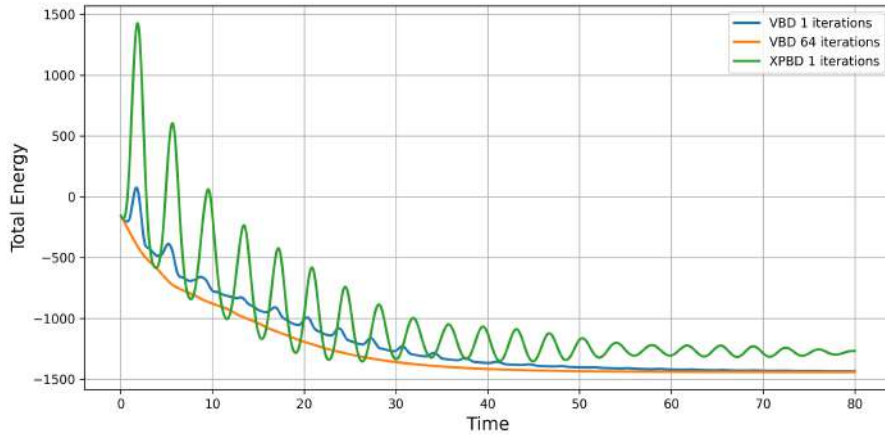


Figure 6.8: Total mechanical energy over a long time window. Both solvers follow the same exponential decay.

Looking more closely (Figure 6.9), $VBD_{it_{\max}1}$ sits between $XPBD_{it1}$ and $XPBD_{it2}$. Both $VBD_{it_{\max}1}$ and $XPBD_{it2}$ already track the reference well but still fluctuate. With 4 iterations both solvers are extremely close to the reference.

6.4 Flexibility

6.4.1 Iteration-count independence of stiffness

For each solver I sweep over a range of iteration counts to check for any noticeable change in stiffness. For both methods the resting configuration is essentially unchanged as the count grows, so the converged shape does not depend on how many iterations are spent.

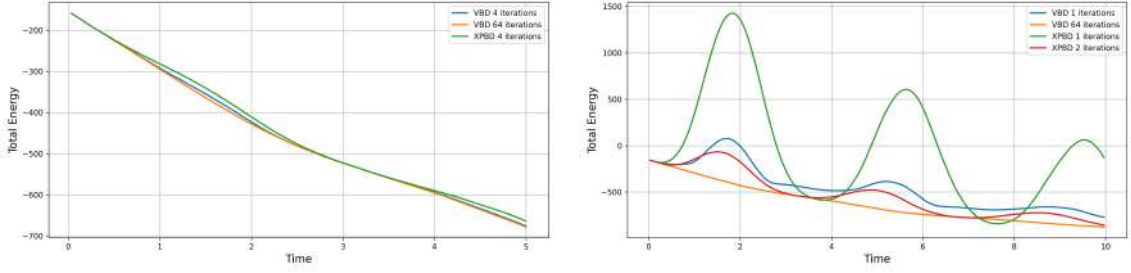


Figure 6.9: Early-time energy for low iteration counts. VBD 1 lies between XPBD 1 and XPBD 2; at 4 iterations both solvers match the reference.

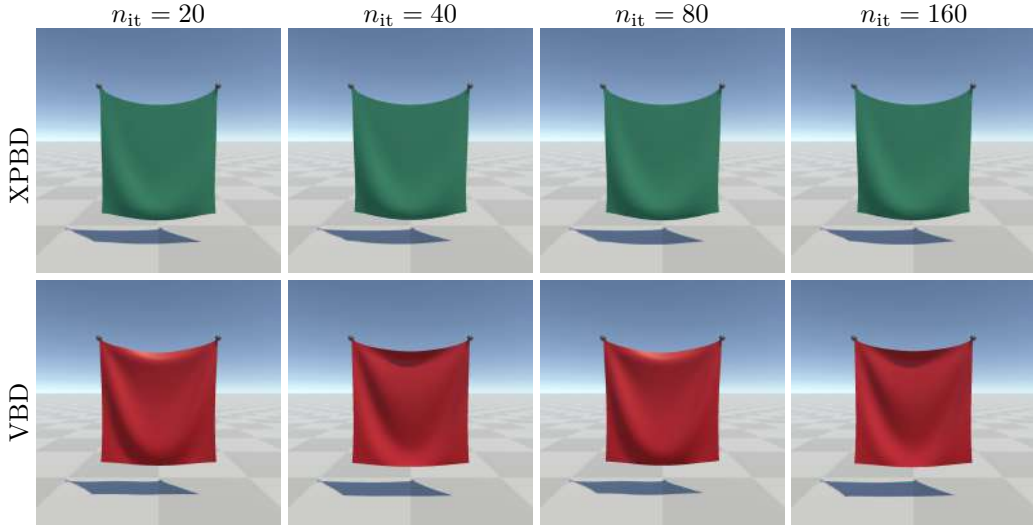


Figure 6.10: Settled shape of a pinned cloth at increasing iteration counts ($n_{it} = 20, 40, 80, 160$). Note that in this experiment the material parameters are different for the two solvers.

6.4.2 Ease of adding a new functionality

To compare how easily each solver can be extended, I had Claude Code (Opus 4.7) implement dihedral bending constraints in both. Both methods already include this constraint in their original papers (XPBD [8], VBD [3]), so I follow those implementations. The existing bending constraint is a distance constraint acting as a skip-spring between two non-adjacent nodes; the task was to implement the dihedral version and make bending toggleable between the dihedral and distance formulations. Table 6.2 summarizes the effort.

The two implementations share a new ≈ 30 -LOC file that computes the gradients from the four hinge-node positions, and the in-solver addition that solves the new constraint is about 28 LOC and essentially identical in both. XPBD was easier to extend: the new constraint resembles the existing distance constraint (but on four particles instead of two), introduces no new concept, and plugs in through a single extra `SolveDihedralConstraints` call without changing any existing code, with each hinge visited once per iteration. VBD needed more setup code because it is vertex-based: each hinge has to be stored four times — once per incident vertex — and its data type carries an extra parameter naming which

of the four vertices is currently being processed. The hinge Hessian block is therefore recomputed four times per iteration, which makes the VBD extension roughly four times as expensive as the XPBD one. Mathematically, XPBD needs only the first derivative of the constraint, whereas VBD needs the second derivative, which is arguably harder to derive and implement.

In general, for scalar constraints — which quantify a condition with a single numerical value, such as a distance or an angle — XPBD is the easier solver to extend.

	XPBD	VBD
Shared file		30 LOC
Solver addition	≈ 28 LOC	≈ 28 LOC
Setup code	≈ 45 LOC	≈ 58 LOC
New data type	4 vertices, rest angle, compliance	+ index of vertex being processed
Structural change	one extra call added	per-vertex; hinge recomputed $4\times/\text{iter}$
Derivative order	first derivative only	first and second derivative

Table 6.2: Cost of adding dihedral bending constraints to each solver.

6.4.3 Coverage of different physical domains

PBD has been around for almost 20 years and XPBD for about 10. These methods have become industry standards for in-game physics. They have been implemented across many domains — cloth, soft bodies, rigid bodies, fluids, hair — with a wide range of constraint types and forces. As a dual solver, XPBD copes well with a wide range of stiffness values, including very stiff and near-rigid constraints (as the stiffness-ratio experiment in Section 6.2.2 shows), which together with its large audience makes it a very flexible solver. This also matched my experience in the test environment: XPBD withstood a wide range of parameters without breaking down, and generally worked out of the box.

VBD, by contrast, is a very new method, published in 2024, and is not yet widely used; its original paper applies it mostly to elastic body dynamics. As the self-collision experiment showed, it is not well suited to very hard (effectively infinite-stiffness) constraints — an issue that AVBD addresses by bringing rigid bodies into the method. In my test environment VBD often had to be tuned — adjusting stiffness ratios or vertex mass relative to stiffness, as discussed in Section 6.2.3 — to stay stable, whereas XPBD tolerated the same configurations without such hand-tuning.

Chapter 7

Discussion

7.1 Synthesis across axes

Reading the results back against the four evaluation axes raises one guiding question per axis. For *efficiency*, which solver dominates the equal-wall-clock frontier, and in which scenario? For *robustness*, where do the stability boundaries diverge, and does that divergence match the primal/dual prediction? For *realism*, which solver’s converged state is closer to Newton at matched compute? And for *flexibility*, how does the cost of extending each solver weigh against its per-frame benefit? The verdicts below answer these questions axis by axis.

7.2 Verdicts across the four axes

Efficiency. On the equal-wall-clock frontier (Section 6.1.1) neither solver dominates out-right; the winner depends on how hard the per-step problem is. A single VBD iteration always lands closer to the Newton reference than a single XPBD iteration, but it costs more. When the time step is large — few substeps, a stiff sub-problem — VBD reaches a given accuracy for less wall-clock, because it spends its budget on iterations that carry second-order information about the energy. As the step shrinks, each sub-problem becomes easy enough for both solvers to converge in one or two iterations, and XPBD’s cheaper per-iteration cost lets it take the frontier. The scaling test (Section 6.1.2) tells the same story at fixed settings: with early stopping, $VBD_{sub4 \times it_{max}4}$ beats XPBD on the easy, low-resolution scenes and converges toward it as the mesh grows, while the cheaper $VBD_{sub4 \times it2}$ configuration outperforms $XPBD_{sub4 \times it4}$ across the board. These figures are single-threaded and therefore understate VBD’s standing (Section 7.3.1).

Robustness. This is the axis where the primal/dual distinction makes its sharpest prediction, and the two stress tests confirm it directly. The mechanisms — why a dual update is slow to propagate a shared-particle mass disparity, and why a per-vertex primal Hessian becomes ill-conditioned under wide stiffness ratios — are derived in Section 2.5; here I only read the results off them. Under a $1000\times$ mass ratio (Section 6.2.1) XPBD — the dual solver — fails to reach the Newton reference within 100 iterations while VBD approaches it. Under a 1:10000 stiffness ratio (Section 6.2.2) the roles reverse: VBD stalls while XPBD matches the reference almost exactly. The self-collision drop (Section 6.2.3) is the same VBD weakness in a practical setting: my 12-spring cloth model carries exactly

the wide stiffness ratios that hurt VBD, so the cloth was unstable unless I raised vertex mass by two-to-three orders of magnitude relative to the largest stiffness. XPBD tolerated these constraints, including near-rigid ones, without hand-tuning. Overall XPBD is the more robust solver for stiff cloth, with the caveat — expanded in Section 7.3.4 — that this VBD failure is specific to the canonical 2024 formulation and is reportedly addressed by AVBD [4].

Physical realism. At matched, sufficient iteration counts the two solvers are essentially indistinguishable from Newton and from each other. The harmonic oscillator (Section 6.3.1) converges in a single iteration for both, leaving only the damping intrinsic to implicit Euler. The energy traces (Section 6.3.2) follow one shared exponential decay; at four iterations both sit on the reference, and the only ordering visible at very low budgets is that one VBD iteration falls between one and two XPBD iterations — the same per-iteration-quality story as the efficiency axis, since the residual oscillations are convergence artifacts rather than a physical difference. Realism is therefore a question of how much iteration budget each solver needs to reach a converged state, not of the converged state itself.

Flexibility. Here the comparison favors each solver on a different sub-question. Both make the converged shape independent of iteration count (Section 6.4.1), so neither couples material stiffness to solver budget. For adding new constraints (Section 6.4.2), XPBD was easier to extend with the scalar dihedral-bending constraint: it needs only the first derivative and a single extra solve call, whereas VBD needs the second derivative and stores each hinge once per incident vertex, recomputing its Hessian block four times an iteration. This advantage is constraint-shaped rather than absolute — for tensor-valued energies such as StVK it would probably be XPBD doing the extra work. On domain coverage (Section 6.4.3) XPBD’s decade of deployment across cloth, fluids, rigid bodies and hair is decisive against a 2024 method still largely confined to elastic bodies.

Joint reading. No solver wins on every axis, so the honest recommendation is scenario-based. VBD is the better choice when the per-step problem is hard — large time steps, heavy mass ratios — and when the target is parallel hardware that suits its per-vertex blocks. XPBD is the better choice when constraints are stiff or near-rigid, when robustness out of the box matters more than peak accuracy per millisecond, or when the breadth of an established ecosystem is an asset. The primal/dual lens earns its place here not as a verdict on which solver is better, but as the thing that predicted — ahead of the measurements — exactly which scenario would break which solver.

Additional remarks. My experiments also reveal a marked difference in *how* the two solvers fail. When XPBD fails, the material becomes overly stretchy but still looks plausible. VBD, by contrast, either shows noticeable artifacts at the point of failure or explodes outright.

7.3 Limitations and threats to validity

7.3.1 Single-threaded execution

All measurements run single-threaded on the CPU. Because VBD parallelizes better than XPBD (Section 5.6), the ms/frame figures are biased against VBD: they understate the advantage it would show in a production, multi-threaded deployment. This is a threat to the external validity of the quantitative efficiency claims only; it leaves the qualitative claims about convergence quality and stability boundaries untouched, since those do not depend on how the work is scheduled across cores. Lifting this limitation — the multithreaded and GPU comparison — is the principal direction of Section 8.2.1.

7.3.2 Mass-spring discretization only

I use distance-spring energies throughout, including for bending. Dihedral bending constraints were implemented (with LLM assistance), but I deliberately did not run the evaluation on this newer model. This choice is not neutral between the solvers: VBD’s original paper uses an energy-based model (StVK), and such a discretization would likely favor VBD to a similar degree that my mass-spring model favors XPBD. The results are therefore not necessarily transferable to other cloth models. The primal/dual framing still carries over — both XPBD and VBD admit arbitrary elastic energies — but the specific stiffness ratios encountered, and hence the magnitude of the stiffness-ratio and per-iteration-convergence effects, may differ. Re-running the comparison on energy-based and alternative-connectivity models is taken up as future work in Section 8.2.3.

7.3.3 Gauss–Seidel order dependence

Both solvers are order-sensitive (Section 2.4). I fix a single deterministic order, but an adversarial reordering could in principle shift the results. I do not investigate this, and it remains a real threat to reproducibility across other implementations.

7.3.4 Canonical 2024 VBD only — the AVBD caveat

The VBD half of the comparison is the canonical 2024 formulation [3]. Augmented VBD [4] targets exactly the failure modes I observe on the stiffness-ratio axis (Section 6.2.2) and the self-collision drop (Section 6.2.3): wide stiffness ratios, near-rigid constraints, and convergence under stiff sub-problems. It is therefore an open question whether the primal-side gap I report is intrinsic to a primal block-coordinate update or specific to the canonical algorithm. I do not resolve this here; re-running the stiffness experiments under AVBD is the follow-up that would settle it, and is listed as future work in Section 8.2.4.

Chapter 8

Conclusion and future work

8.1 Summary

This thesis presented a controlled comparison of XPBD — one of the most established solvers for real-time deformable simulation — and VBD, a primal alternative introduced in 2024. The contribution is threefold: a shared Unity codebase that implements both solvers under matched material parameters and a matched compute budget; a Newton ground-truth reference that enables reporting absolute solution error rather than only relative performance; and an evaluation along four axes — efficiency, robustness, physical realism, and flexibility — read against the primal/dual distinction that motivated the stress tests.

The headline finding is that neither solver dominates; each wins in the regime its update direction is suited to. On the equal-wall-clock frontier VBD leads when the per-step problem is hard (large time steps), while XPBD overtakes it once finer substepping makes each sub-problem easy enough to converge in one or two cheap iterations. Robustness splits exactly as the primal/dual lens predicts: XPBD, the dual solver, handles extreme stiffness ratios and near-rigid constraints but stalls under a $1000\times$ mass ratio; VBD, the primal solver, handles the mass ratio but becomes unstable under the wide stiffness ratios of a realistic cloth model unless its vertex mass is tuned. At sufficient iterations both reach the Newton reference and are physically indistinguishable, so realism reduces to a per-iteration-quality question already captured by the efficiency axis. On flexibility, XPBD is easier to extend with scalar constraints and benefits from a decade of deployment across many domains, whereas VBD’s per-vertex structure is heavier to extend but parallelizes more cleanly. The practical recommendation is therefore scenario-based rather than absolute, and is set out in full in Section 7.2.

8.2 Future work

8.2.1 Multithreaded and GPU comparison

The most consequential extension is to repeat the experiments on a multithreaded CPU and on the GPU, directly lifting the single-threaded limitation of Section 7.3.1. Both solvers admit parallelization through graph coloring, but their structures differ in granularity (Section 5.6): VBD’s per-vertex blocks should color more cleanly than XPBD’s per-

constraint blocks. The expected outcome is therefore that parallelization widens VBD’s lead, and quantifying that gap is the natural next study.

8.2.2 Extending the research to other domains of dynamics

Beyond mass-spring systems, both solvers could in theory also handle other cloth representations, soft bodies, and rigid bodies. It would be valuable to see how the two perform on these other kinds of dynamics, and whether the same conclusions continue to hold.

8.2.3 Using a different cloth model

The mass-spring discretization is the source of the bias discussed in Section 7.3.2, which makes the cloth model the most informative ingredient to vary next. The cheapest change is to replace my twelve-neighbor-per-vertex model with a four- or eight-neighbor one; this narrows the range of representable materials but would reveal how sensitive the comparison is to mesh connectivity. A more substantial change is to swap the mass-spring energy for an energy-based model such as Saint-Venant–Kirchhoff (StVK), the model the original VBD paper uses; I would expect it to favor XPBD far less. A particularly informative variant would pair the models asymmetrically — mass-spring XPBD against StVK VBD — letting each solver run on the discretization that suits it best.

8.2.4 AVBD and post-2024 variants

Augmented VBD [4] reportedly handles wide stiffness ratios and high stiffness substantially better than canonical VBD. Re-running the stiffness-ratio experiments with AVBD would resolve the question left open in Section 7.3.4: whether the gap I observe is intrinsic to the primal approach or an artifact of the canonical 2024 implementation.

Acknowledgements

I would like to express my sincere gratitude to my advisor, dr. Márton Vaitkus, for his guidance and prompt feedback throughout the preparation of this thesis. I am also deeply grateful to my family for their support during my university studies, and to my friends for making these years memorable.

Bibliography

- [1] David Baraff and Andrew Witkin. Large steps in cloth simulation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '98, page 43–54, New York, NY, USA, 1998. Association for Computing Machinery. ISBN 0897919998. DOI: 10.1145/280814.280821. URL <https://doi.org/10.1145/280814.280821>.
- [2] Anka He Chen. Tinyvbd. <https://github.com/AnkaChan/TinyVBD>, 2024.
- [3] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. Vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2024)*, 43(4):116:1–116:16, 07 2024. ISSN 0730-0301. DOI: 10.1145/3658179. URL <https://doi.org/10.1145/3658179>.
- [4] Chris Giles, Elie Diaz, and Cem Yuksel. Augmented vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2025)*, 44(4), 08 2025. ISSN 0730-0301. DOI: 10.1145/3731195. URL <https://doi.org/10.1145/3731195>.
- [5] Minchen Li, Chenfanfu Jiang, Zhaofeng Luo, Wenxin Du, Chang Yu, Žiga Kovačič, and Tianyi Xie. *Physics-Based Simulation*. Self-published, March 2026. URL <https://phys-sim-book.github.io/>.
- [6] Jun Ma, Nak-Jun Sung, Min-Hyung Choi, and Min Hong. Performance comparison of vertex block descent and position based dynamics algorithms using cloth simulation in unity. *Applied Sciences*, 14(23), 2024. ISSN 2076-3417. DOI: 10.3390/app142311072. URL <https://www.mdpi.com/2076-3417/14/23/11072>.
- [7] M. Macklin, K. Erleben, M. Müller, N. Chentanez, S. Jeschke, and T. Y. Kim. Primal/dual descent methods for dynamics. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA '20, Goslar, DEU, 2020. Eurographics Association. DOI: 10.1111/cgf.14104. URL <https://doi.org/10.1111/cgf.14104>.
- [8] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, MIG '16, page 49–54, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450345927. DOI: 10.1145/2994258.2994272. URL <https://doi.org/10.1145/2994258.2994272>.
- [9] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. *Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2019. URL <https://api.semanticscholar.org/CorpusID:197928472>.

- [10] Matthias Müller. Ten minute physics. <https://matthias-research.github.io/pages/tenMinutePhysics/index.html>, 2022.
- [11] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. ISSN 1047-3203. DOI: <https://doi.org/10.1016/j.jvcir.2007.01.005>. URL <https://www.sciencedirect.com/science/article/pii/S1047320307000065>.
- [12] G. P. Nikishkov. *Introduction to the Finite Element Method*. University of Aizu, Aizu-Wakamatsu, Japan, 2004.
- [13] Huamin Wang. A chebyshev semi-iterative approach for accelerating projective and position-based dynamics. *ACM Trans. Graph.*, 34(6), November 2015. ISSN 0730-0301. DOI: 10.1145/2816795.2818063. URL <https://doi.org/10.1145/2816795.2818063>.

Appendix

A.1 Declaration on the Use of Generative Artificial Intelligence

☐ I have not used any generative AI tools.

☒ I have used generative AI tools. I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	Name of Generative AI Tool(s)	Affected Sections (chapter, page number, reference)	Estimate Proportion of Use (per usage type)
Literature Review	Gemini 3.1, Claude Opus 4.7	Chapters 2, 3, 4	30%
Brief Summary of the Prompt	searching literature on the comparison of XPBD and VBD; help in understanding core concepts of the theory		
Program Code Generation	Claude Opus 4.7	Chapter 6	35%
Brief Summary of the Prompt	implementing Rayleigh damping, dihedral constraints from the original papers creating the chain model based on the cloth model; debugging simulation instabilities; cross-checking the implementation against the original papers		
Generating New Ideas or Solution Proposals	Claude Opus 4.7	Chapter 6	30%
Brief Summary of the Prompt	suggesting experiments worth running for the XPBD-VBD comparison		
Creating an Outline (text structure, bullet points)	Claude Opus 4.7	All chapters	26%
Brief Summary of the Prompt	drafting a thesis outline for the comparison of XPBD and VBD based on the implemented solvers and collected material		
Creating Text Blocks	Claude Opus 4.7, Gemini Flash (NotebookLM)	All chapters	43%
Brief Summary of the Prompt	turning bullet-point notes into running text; refining wording to academic US English		
Generating Images for Illustrative Purposes	–	–	–
Brief Summary of the Prompt	–		
Data Visualization, Generating Charts Based on Data Points	Claude Opus 4.7	Chapter 6	36%
Brief Summary of the Prompt			
Preparing a Presentation	–	–	–
Brief Summary of the Prompt	writing visualization scripts in Python		
Other (please specify)	–	–	–
Brief Summary of the Prompt	–		
Aggregated Percentage Value (for the core part of the task):			28%
Brief Textual Justification of the Aggregated Value:			
Generative AI was mainly used for supporting tasks – literature surveying, code scaffolding, experiment brainstorming, outline structuring, and language polishing – where they substantially accelerated the work. The core content of the thesis remains my own: implementing the solvers and integrating them into Unity, designing and executing the comparative experiments, analyzing the measured data, and drawing the conclusions. The aggregated 28% reflects this balance between AI-accelerated secondary work and the technical and analytical core authored by me.			

A.2 Full solver pseudocode

Chapter 4 states both solvers in their stripped core-comparison form. This appendix gives the more detailed, full versions as actually implemented (Chapter 5): with Rayleigh damping, self-collision, and (for VBD) Chebyshev acceleration. Here I provide the original early-stopping variant. Optional terms are guarded by flags and disabled for the core comparison.

A.2.1 XPBD, full implementation

```
// Initialize cloth mesh, rest positions, inverse masses, distance constraints, parameters

Step(dt):
    sdt = dt / numSubsteps

    spatialHash.Create(positions) // build spatial hash once per frame
    maxTravel = thickness + maxVelocity * dt
    spatialHash.QueryAll(positions, maxTravel)

    for step = 1 to numSubsteps:
        AccumulateExternalForces() // e.g. wind, refreshed each substep
        Integrate(sdt)
        ResetLambdas()
        for iter = 1 to numIterations:
            SolveDistanceConstraints(sdt)
            SolveSelfCollisions()
            SolveExternalCollisions()
            UpdateVelocities(sdt)

Integrate(sdt):
    for each vertex i with invMasses[i] != 0:
        velocities[i] += gravity * sdt
        velocities[i] += externalForces[i] * sdt * invMasses[i]
        velocities[i] /= 1 + rayleighMassDamping * sdt // mass damping
        if |velocities[i]| > maxVelocity:
            velocities[i] *= maxVelocity / |velocities[i]| // velocity capping
        previousPositions[i] = positions[i]
        positions[i] += velocities[i] * sdt

SolveDistanceConstraints(sdt):
    for each constraint c = (id0, id1, restLength, compliance):
        w0 = invMasses[id0]; w1 = invMasses[id1]; w = w0 + w1
        if w == 0: continue
        grad = positions[id0] - positions[id1]
        len = |grad|; if len == 0: continue
        grad /= len
        C = len - restLength
        alphaTilde = compliance / sdt^2 // time-step-scaled compliance
        gamma = compliance * rayleighStiffnessDamping / sdt // stiffness damping
        dx = (positions[id0]-previousPositions[id0]) - (positions[id1]-previousPositions[id1])
        deltaLambda = (-C - alphaTilde*lambdas[c] - gamma*(grad · dx)) / ((1 + gamma)*w + alphaTilde)
    )
    lambdas[c] += deltaLambda
    positions[id0] += grad * (deltaLambda * w0)
    positions[id1] -= grad * (deltaLambda * w1)

SolveSelfCollisions():
    for each vertex id0 with invMasses[id0] != 0:
        for each id1 in spatialHash.Adjacent(id0):
            w0 = invMasses[id0]; w1 = invMasses[id1]; w = w0 + w1
            if w == 0: continue
            minDist = min(thickness, |restPositions[id0] - restPositions[id1]|)
            delta = positions[id0] - positions[id1]
            dist = |delta|
            if dist == 0 or dist >= minDist: continue
            normal = delta / dist
            correction = normal * (minDist - dist)
            positions[id0] += correction * (w0 / w)
            positions[id1] -= correction * (w1 / w)
            relDisp = (positions[id0]-previousPositions[id0]) - (positions[id1]-previousPositions[
```

```

        dispTangent = relDisp - (relDisp · normal) * normal
        positions[id0] -= dispTangent * (selfCollisionFriction * w0 / w)
        positions[id1] += dispTangent * (selfCollisionFriction * w1 / w)

SolveExternalCollisions():
    // ground & sphere generalized
    for each vertex i with invMasses[i] != 0:
        for each collider:
            if collider.Penetrates(positions[i]):
                normal = collider.SurfaceNormal(positions[i])
                positions[i] += normal * collider.PenetrationDepth(positions[i])
                // tangential friction against previousPositions[i]

UpdateVelocities(sdt):
    for each vertex i with invMasses[i] != 0:
        velocities[i] = (positions[i] - previousPositions[i]) / sdt

```

Listing A.2.1: XPBD step as implemented.

A.2.2 VBD, full implementation

```

// Initialize cloth mesh, rest positions, masses, per-vertex spring adjacency, parameters

Step(dt):
    sdt = dt / numSubsteps

    spatialHash.Create(positions) // build spatial hash once per frame
    maxTravel = thickness + maxVelocity * dt
    spatialHash.QueryAllSymmetric(positions, maxTravel)

    for step = 1 to numSubsteps:
        AccumulateExternalForces()
        AdaptiveInitialization(sdt)
        ClearIsColliding()
        omega = 1
        for iter = 1 to maxIterations:
            prevIterPositions = positions
            maxStep = Solve(sdt)
            omega = GetAcceleratorOmega(iter, accelerationRho, omega) // Chebyshev acceleration
            ApplyAccelerator(omega)
            prevPrevIterPositions = prevIterPositions
            if maxStep < convergenceTolerance: break // early-out if converged
            UpdateVelocities(sdt)

AdaptiveInitialization(sdt):
    previousPositions = positions
    for each vertex i with invMasses[i] != 0:
        aExt = gravity + externalForces[i] * invMasses[i]
        inertia[i] = previousPositions[i] + sdt*velocities[i] + sdt^2 * aExt
        prevAcc = (velocities[i] - previousVelocities[i]) / sdt
        aTilde = Clamp01( (prevAcc · aExt) / |aExt|^2 ) // adaptive warm start
        positions[i] = previousPositions[i] + sdt*velocities[i] + aTilde*sdt^2*aExt

Solve(sdt):
    // one Gauss-Seidel block-descent pass
    maxStep2 = 0
    for each vertex i with invMasses[i] != 0:
        m_h2 = masses[i] / sdt^2
        f = (inertia[i] - positions[i]) * m_h2 // inertial force
        H = m_h2 * Identity3x3 // inertial Hessian (symmetric)

        cMass = rayleighMassDamping * masses[i] / sdt // mass damping
        f -= cMass * (positions[i] - previousPositions[i])
        H += cMass * Identity3x3

        for each spring edge (i, j) with restLength l0, stiffness k:
            diff = positions[i] - positions[j]
            len = |diff|; if len < eps: continue
            d = diff / len; ratio = l0 / len
            coeff1 = k * max(0, 1 - ratio) // compression-clamped (PSD) Hessian
            coeff2 = k * ratio
            dv = (positions[i]-previousPositions[i]) - (positions[j]-previousPositions[j])

```

```

        betaOverDt = rayleighStiffnessDamping / sdt          // stiffness damping
        f -= betaOverDt * (coeff1*dv + coeff2*(d · dv)*d)
        coeff1 *= (1 + betaOverDt); coeff2 *= (1 + betaOverDt)
        H += coeff1 * Identity3x3 + coeff2 * (d * d^T)
        f += (k * (10 - len) / len) * diff

    for each j in spatialHash.Adjacent(i):                    // self-collision handling
        diff = positions[i] - positions[j]
        dist = |diff|; if dist < eps: continue
        minDist = min(thickness, |restPositions[i] - restPositions[j]|)
        if dist >= minDist: continue
        n = diff / dist
        f += (selfCollisionStiffness * (minDist - dist)) * n    // penalty contact force
        H += selfCollisionStiffness * (n * n^T)
        isColliding[i] = true

    AddExternalCollisions(i, f, H)                            // ground/sphere penalties

    dx = Solve3x3(H, f)                                       // local Newton step, H dx = f
    positions[i] += dx
    maxStep2 = max(maxStep2, |dx|^2)
    return sqrt(maxStep2)

GetAcceleratorOmega(order, rho, prevOmega):                  // Chebyshev-acceleration parameter
    if order == 1: return 1
    if order == 2: return 2 / (2 - rho^2)
    return 4 / (4 - rho^2 * prevOmega)

ApplyAccelerator(omega):
    if omega <= 1: return
    for each vertex i with invMasses[i] != 0 and not isColliding[i]:
        positions[i] = omega * (positions[i] - prevPrevIterPositions[i]) + prevPrevIterPositions[i]

UpdateVelocities(sdt):
    previousVelocities = velocities
    for each vertex i:
        if invMasses[i] == 0: velocities[i] = 0; continue
        v = (positions[i] - previousPositions[i]) / sdt
        if |v| > maxVelocity: v *= maxVelocity / |v|          // velocity capping
        velocities[i] = v

```

Listing A.2.2: VBD step as implemented.